



Nome :

Data:

Atividade Presencial (AP1) com consulta teórica

Entrega pelo SIGAA - Valor: 4.0 pontos

1. Faça um programa que leia três valores inteiros (A,B,C) e chame uma função que receba estes 3 valores de entrada e retorne no valor A soma dos valores (A+B+C), no valor B a multiplicação dos valores (B*C) e no valor C, o maior entre A, B e C. Após a execução da função exibir os valores de A,B,C e os valores calculados pela função. (Valor 1.0 pontos)

```
typedef struct data{
    int dia,mes,ano;
}Data;

typedef struct pessoa{
    char nome[20];
    int idade;
    Data dNascimento;
}Pessoa;
```

Figura 1: Estrutura Data e Pessoa

2. Com base nas estruturas Data e Pessoa (Figura 1), implemente um programa para: (Valor 2.0)
 - A) Instanciar três novas pessoas, para cada pessoa solicite o seu nome e a data de nascimento, formada por dia, mês e ano.
 - B) Solicite uma data atual, formada por dia,mês e ano, armazene em uma variável do tipo Data.
 - C) Implemente uma função para receber um ponteiro de Pessoa e uma data (data atual). A função deve calcular a idade da pessoa (Data Atual – Data Nascimento) e atribuir a idade no campo “idade”, na struct pessoa.

```
void defineDataNascimento(Pessoa *ps, Data dAtual);
```

- D) Crie um vetor de pessoas (vPessoas), atribua cópia das três pessoas a posições do vetor, após passe o vetor para uma função que deverá mostrar as pessoas em ordem crescente de idade (mostre o nome, a idade e a data de nascimento de cada uma);

3. Com base nas estruturas Data e Pessoa (Figura 1), implemente uma lista simplesmente encadeada (LSE) de Pessoas. Após implemente: (valor 2.0 pontos)

A) Um função para alocar 6 pessoas com dados aleatórios de nome e datas de nascimento, após os insira na lista simplesmente encadeada recebida por referência.

```
// A) Função para criar 6 pessoas aleatórias
void criar_pessoas_aleatorias(Lista *lista) {
```

B) Crie uma função para percorrer todos elementos da lista mostrando os dados de cada pessoa;

```
// B) Função para percorrer e mostrar a lista
void mostrar_lista(Lista lista) {
```

C) Defina a função cópia de lista Ordenada, a função recebe lista de pessoas (original - passada por valor) e um ponteiro de lista (*listaOrdenada). A função faz a cópia dos elementos da lista original para lista ordenada, de modo ordenado por idade. Após, mostre a lista Ordenada.

```
// C) Função para copiar lista ordenada por idade
void copiar_lista_ordenada(Lista original, Lista *listaOrdenada) {
```

4. Com base nas estruturas Data e Pessoa (Figura 1), e nas estruturas Nó e Lista (Figura 2) referente a lista duplamente encadeada (LDE) de Pessoas. Após avalie as questões: (valor 1.0 Extra)

```
typedef struct no {
    Pessoa pessoa;
    struct no *anterior;
    struct no *proximo;
} No;

typedef struct lista {
    No *inicio;
    No *fim;
    int tamanho;
} Lista;
```

*Figura 2: Estruturas Nó e Lista
Duplamente Encadeada*

- A) Defina um nome para a função (nome_função), explique objetivo principal da função.
 B) Comente o código das linhas (6, 9, 13,17) .
 C) As linhas 22 e 23 podem ser trocadas de ordem? Justifique sua resposta.
 D) As linhas 24 e 25 podem ser trocadas de ordem? Justifique sua resposta.

```

1  void nome_funcao(Lista *lista, Pessoa p) {
2      calcular_idade(&p);
3      No *novo = (No*)malloc(sizeof(No));
4      novo->pessoa = p; novo->anterior = NULL; novo->proximo = NULL;
5
6      if (lista->inicio == NULL) {
7          lista->inicio = novo;
8          lista->fim = novo;
9      }else if (strcmp(p.nome, lista->inicio->pessoa.nome) < 0) {
10         novo->proximo = lista->inicio;
11         lista->inicio->anterior = novo;
12         lista->inicio = novo;
13     }else if (strcmp(p.nome, lista->fim->pessoa.nome) > 0) {
14         novo->anterior = lista->fim;
15         lista->fim->proximo = novo;
16         lista->fim = novo;
17     }else {
18         No *atual = lista->inicio;
19         while (atual != NULL && strcmp(p.nome, atual->pessoa.nome) > 0) {
20             atual = atual->proximo;
21         }
22         novo->anterior = atual->anterior;
23         novo->proximo = atual;
24         atual->anterior->proximo = novo;
25         atual->anterior = novo;
26     }
27     lista->tamanho++;
28 }

```

Enviar para o SIGAA as soluções desenvolvidas nos exercícios de 1 a 3(arquivos “.c”). A questão 4 deverá respondida em um arquivo texto. Os arquivos deverão ser compactados em um arquivo “Ap1_nomeAluno.xxx”.

Bom Avaliação!!!